Separation Logic I

The heap, the separating conjunction, and the frame rule

Cristina David

Lecture 10 of the verification half-unit

COMSM0067 — Advanced Topics in Programming Languages
University of Bristol

Where we are

Two lectures of Hoare logic. It reasons beautifully about variables.

But variables are independent boxes: assigning to x cannot possibly change y. That is what made the assignment axiom a *substitution*.

Where we are

Two lectures of Hoare logic. It reasons beautifully about variables.

But variables are independent boxes: assigning to x cannot possibly change y. That is what made the assignment axiom a *substitution*.

Today we add the heap — and that assumption dies.

Why the assignment axiom breaks

Add pointers. Now $[x]$ means "the heap cell that x points to".

Consider the mutation $[x] := 4$, and try to reason with substitution as before.

Why the assignment axiom breaks

Add pointers. Now $[x]$ means "the heap cell that x points to".

Consider the mutation $[x] := 4$, and try to reason with substitution as before.

The problem: aliasing

If x and y hold the *same* address, then $[x] := 4$ also changes $[y]$.

Substitution into an assertion about $[y]$ cannot see this — the syntax gives no hint that x and y are the same cell.

Why the assignment axiom breaks

Add pointers. Now $[x]$ means "the heap cell that x points to".

Consider the mutation $[x] := 4$, and try to reason with substitution as before.

The problem: aliasing

If x and y hold the *same* address, then $[x] := 4$ also changes $[y]$.

Substitution into an assertion about $[y]$ cannot see this — the syntax gives no hint that x and y are the same cell.

The assignment axiom worked because a variable name *is* its location. A pointer's name is not its location: many names, one cell.

The real question

How do we reason about a program that mutates a shared, aliased heap — locally?

The real question

How do we reason about a program that mutates
a shared, aliased heap — locally?

"Locally" is the hard word. We want to reason about the cells a command *touches*, and ignore the rest of the heap — without proving, every time, that "the rest" really is untouched.

Separation logic is the machinery that makes "ignore the rest" sound.

The model

States, now with a heap

A state is now two components:

$$\text{store } s : \mathrm{Var} \to \mathrm{Val} \qquad \text{heap } h : \mathrm{Loc} \rightharpoonup \mathrm{Val}$$

- The store is the old σ: variables to values. Total.
- The heap maps *locations* to values, and is partial — its domain $\mathrm{dom}(h)$ is the finite set of currently-allocated cells.

States, now with a heap

A state is now two components:

$$\text{store } s : \mathrm{Var} \to \mathrm{Val} \qquad\qquad \text{heap } h : \mathrm{Loc} \rightharpoonup \mathrm{Val}$$

- The store is the old σ: variables to values. Total.
- The heap maps *locations* to values, and is partial — its domain $\mathrm{dom}(h)$ is the finite set of currently-allocated cells.

We write $s, h \vDash P$. The novelty is entirely in h: assertions will now describe the shape of the heap, not just the values of variables.

Heap assertions

The empty heap

$s, h \vDash \mathsf{emp} \quad \text{iff} \quad \mathrm{dom}(h) = \varnothing.$

Points-to: a single cell

$s, h \vDash x \mapsto v \quad \text{iff} \quad \mathrm{dom}(h) = \{\, s(x) \,\} \quad \text{and} \quad h(s(x)) = s(v).$

Heap assertions

The empty heap

$s, h \vDash \mathsf{emp} \quad \text{iff} \quad \mathrm{dom}(h) = \varnothing.$

Points-to: a single cell

$s, h \vDash x \mapsto v \quad \text{iff} \quad \mathrm{dom}(h) = \{\, s(x) \,\} \quad \text{and} \quad h(s(x)) = s(v).$

Read $x \mapsto v$ as: "the heap is exactly one cell, at address x, holding v".

Not "x points to v somewhere in a bigger heap" — exactly one cell. This precision is the point, as the next slide shows.

The separating conjunction

$P * Q$ — "P and Q, on disjoint heaps"

$s, h \vDash P * Q$ iff h splits as $h = h_1 \uplus h_2$ with

$$h_1 \perp h_2, \qquad s, h_1 \vDash P, \qquad s, h_2 \vDash Q.$$

($\uplus$ is union of heaps with disjoint domains.)

The separating conjunction

$P * Q$ — "P and Q, on disjoint heaps"

$s, h \vDash P * Q$ iff h splits as $h = h_1 \uplus h_2$ with

$$h_1 \perp h_2, \qquad s, h_1 \vDash P, \qquad s, h_2 \vDash Q.$$

($\uplus$ is union of heaps with disjoint domains.)

$P * Q$ asserts P and Q hold on separate parts of the heap. The split is an *existential*: "there is some way to divide the heap so that…".

This one connective is the whole idea. Everything else follows.

$*$ is not $\wedge$

$$x \mapsto 3 \ * \ y \mapsto 3 \qquad \text{versus} \qquad x \mapsto 3 \ \wedge \ y \mapsto 3$$

$*$ is not $\wedge$

$$x \mapsto 3 \ * \ y \mapsto 3 \qquad \text{versus} \qquad x \mapsto 3 \ \wedge \ y \mapsto 3$$

- $x \mapsto 3 * y \mapsto 3$: two disjoint cells, so $x \neq y$ is forced. Two separate boxes, both holding 3.

$*$ is not $\wedge$

$$x \mapsto 3 \ * \ y \mapsto 3 \qquad \text{versus} \qquad x \mapsto 3 \ \wedge \ y \mapsto 3$$

- $x \mapsto 3 * y \mapsto 3$: two disjoint cells, so $x \neq y$ is forced. Two separate boxes, both holding 3.
- $x \mapsto 3 \wedge y \mapsto 3$: *one* heap satisfying both. A single-cell heap does, if $x = y$. So this permits aliasing.

$*$ is not $\wedge$

$$x \mapsto 3 \ * \ y \mapsto 3 \qquad \text{versus} \qquad x \mapsto 3 \ \wedge \ y \mapsto 3$$

- $x \mapsto 3 * y \mapsto 3$: two disjoint cells, so $x \neq y$ is forced. Two separate boxes, both holding 3.
- $x \mapsto 3 \wedge y \mapsto 3$: *one* heap satisfying both. A single-cell heap does, if $x = y$. So this permits aliasing.

The slogan

$*$ builds in disjointness. Writing $P * Q$ is a *claim* that the two footprints do not overlap — and that claim is exactly what lets us reason locally.

Small axioms

Commands for the heap

Four new commands, each with a small axiom — stated on the smallest heap that makes sense, its *footprint*.

mutation	$\{x \mapsto _\}\ \ [x] := a\ \ \{x \mapsto a\}$	
lookup	$\{x \mapsto v\}\ \ y := [x]\ \ \{y = v \wedge x \mapsto v\}$	(y fresh)
allocation	$\{\mathsf{emp}\}\ \ x := \mathsf{cons}(a)\ \ \{x \mapsto a\}$	
disposal	$\{x \mapsto _\}\ \ \mathsf{dispose}(x)\ \ \{\mathsf{emp}\}$	

Commands for the heap

Four new commands, each with a small axiom — stated on the smallest heap that makes sense, its *footprint*.

$$\begin{array}{rl} \text{mutation} & \{x \mapsto _\}\ \ [x] := a\ \ \{x \mapsto a\} \\ \text{lookup} & \{x \mapsto v\}\ \ y := [x]\ \ \{y = v \wedge x \mapsto v\} \qquad (y \text{ fresh}) \\ \text{allocation} & \{\text{emp}\}\ \ x := \text{cons}(a)\ \ \{x \mapsto a\} \\ \text{disposal} & \{x \mapsto _\}\ \ \text{dispose}(x)\ \ \{\text{emp}\} \end{array}$$

Each mentions only the cell it touches. Mutation's precondition is a single cell $x \mapsto _$ — nothing about the rest of the heap. But real programs run in a big heap. How do we scale these up?

The frame rule

The frame rule

$$\frac{\{P\}\ c\ \{Q\}}{\{P * R\}\ c\ \{Q * R\}}\ \textsf{Frame} \qquad \mathrm{mod}(c) \cap \mathrm{fv}(R) = \varnothing$$

The frame rule

$$\frac{\{P\}\ c\ \{Q\}}{\{P * R\}\ c\ \{Q * R\}}\ \text{Frame} \qquad \mathrm{mod}(c) \cap \mathrm{fv}(R) = \varnothing$$

In words: if c transforms P into Q using only the footprint P, then it leaves any disjoint heap R untouched.

The frame rule

$$\frac{\{P\}\ c\ \{Q\}}{\{P * R\}\ c\ \{Q * R\}}\ \text{Frame} \qquad \mathrm{mod}(c) \cap \mathrm{fv}(R) = \varnothing$$

In words: if c transforms P into Q using only the footprint P, then it leaves any disjoint heap R untouched.

This is the heart of separation logic.

Prove a command correct *once*, on its small footprint. The frame rule lets you drop it into *any* larger heap for free — R comes along unchanged.

The frame rule at work

The small axiom for mutation:

$$\{x \mapsto _\}\ \ [x] := a\ \ \{x \mapsto a\}$$

The frame rule at work

The small axiom for mutation:

$$\{x \mapsto _\}\ \ [x] := a\ \ \{x \mapsto a\}$$

Frame on an arbitrary disjoint rest-of-heap R:

$$\{(x \mapsto _) * R\}\ \ [x] := a\ \ \{(x \mapsto a) * R\}$$

The frame rule at work

The small axiom for mutation:

$$\{x \mapsto _\}\ \ [x] := a\ \ \{x \mapsto a\}$$

Frame on an arbitrary disjoint rest-of-heap R:

$$\{(x \mapsto _) * R\}\ \ [x] := a\ \ \{(x \mapsto a) * R\}$$

R might be a million other cells, a whole data structure. We never mention them, never re-verify them — the frame rule certifies, *once and for all*, that $[x] := a$ cannot touch them, because $*$ guaranteed they were disjoint.

Local axioms + framing = modular reasoning about the heap.

The side condition is not optional

$$\frac{\{P\}\ c\ \{Q\}}{\{P * R\}\ c\ \{Q * R\}} \qquad \mathrm{mod}(c) \cap \mathrm{fv}(R) = \varnothing$$

Drop it, and take $c \equiv (x := 4)$ — an ordinary variable assignment, so $\mathrm{mod}(c) = \{x\}$.

A true triple: $\{\mathsf{emp}\}\ x := 4\ \{x = 4\}$.

The side condition is not optional

$$\frac{\{P\}\ c\ \{Q\}}{\{P * R\}\ c\ \{Q * R\}} \qquad \mathrm{mod}(c) \cap \mathrm{fv}(R) = \varnothing$$

Drop it, and take $c \equiv (x := 4)$ — an ordinary variable assignment, so $\mathrm{mod}(c) = \{x\}$.

A true triple: $\{\mathsf{emp}\}\ x := 4\ \{x = 4\}$. Frame with $R \equiv (x = 0)$, which mentions x:

$$\{\mathsf{emp} * (x = 0)\}\ \ x := 4\ \ \{(x = 4) * (x = 0)\}\ \ =\ \ \{x = 0\}\ \ x := 4\ \ \{x = 4 \wedge x = 0\}$$

The side condition is not optional

$$\frac{\{P\}\ c\ \{Q\}}{\{P * R\}\ c\ \{Q * R\}} \qquad \mathrm{mod}(c) \cap \mathrm{fv}(R) = \varnothing$$

Drop it, and take $c \equiv (x := 4)$ — an ordinary variable assignment, so $\mathrm{mod}(c) = \{x\}$.

A true triple: $\{\mathsf{emp}\}\ x := 4\ \{x = 4\}$. Frame with $R \equiv (x = 0)$, which mentions x:

$$\{\mathsf{emp} * (x = 0)\}\ \ x := 4\ \ \{(x = 4) * (x = 0)\}\ \ =\ \ \{x = 0\}\ \ x := 4\ \ \{x = 4 \wedge x = 0\}$$

False. The frame R talked about a variable the command *changes*.

The side condition protects exactly this: R must not mention what c modifies.

Why $\wedge$ could never do this

Could we state a frame rule with $\wedge$ instead of $*$?

$$\{P\}\ c\ \{Q\} \implies \{P \wedge R\}\ c\ \{Q \wedge R\} \qquad \text{no}$$

Why $\wedge$ could never do this

Could we state a frame rule with $\wedge$ instead of $*$?

$$\{P\}\ c\ \{Q\} \implies \{P \wedge R\}\ c\ \{Q \wedge R\} \qquad \text{no}$$

$\wedge$ does not force R's heap to be disjoint from P's. So R might describe the *very cell* c mutates, and then c falsifies R — the aliasing problem, back again.

Why $\wedge$ could never do this

Could we state a frame rule with $\wedge$ instead of $*$?

$$\{P\}\ c\ \{Q\} \implies \{P \wedge R\}\ c\ \{Q \wedge R\} \qquad \text{no}$$

$\wedge$ does not force R's heap to be disjoint from P's. So R might describe the *very cell* c mutates, and then c falsifies R — the aliasing problem, back again.

$*$ earns its keep here: it is the connective for which framing is sound. $\wedge$ is not.

This is why Hoare logic could not simply be "extended" to the heap — the conjunction was the wrong one.

Wrap

Today

- Variables are boxes; heap cells are not — aliasing kills the assignment axiom's substitution trick.
- State = store + partial heap. Assertions describe heap shape: emp, $x \mapsto v$.
- The separating conjunction $P * Q$ splits the heap into disjoint parts. $* \neq \wedge$: $*$ *builds in disjointness*.
- Small axioms on footprints + the frame rule = local, modular reasoning. The side condition is load-bearing.

Next time

We can describe a single cell. Real programs use data structures — lists, trees — which are unboundedly many cells.

We need recursive assertions: $\mathsf{list}(x)$, $\mathsf{lseg}(x, y)$.

Next time

We can describe a single cell. Real programs use data structures — lists, trees — which are unboundedly many cells.

We need recursive assertions: $\mathsf{list}(x)$, $\mathsf{lseg}(x, y)$.

Separation Logic II (Lecture 11): inductive predicates, and the proof that pays for all this machinery — in-place list reversal.

Problem Sheet 10 is out: $*$ vs $\wedge$, heap models, and the frame rule (including what breaks without the side condition).

Questions?